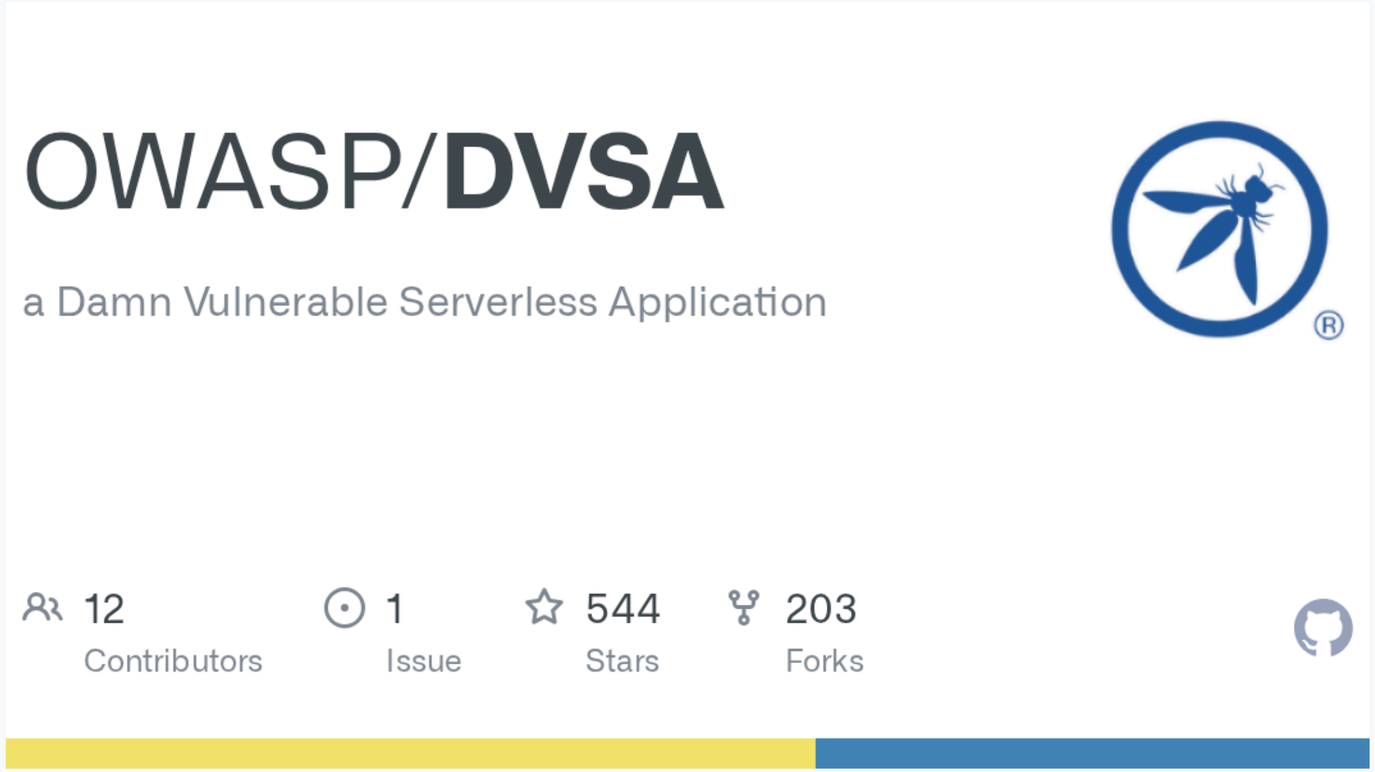
ICS 344 Project

DVSA - Vulnerabilities & Fixes



OFFICIAL LESSONS

10

exploited, fixed, verified

CORE PLATFORM

AWS

DVSA serverless deployment

Members

Muhammad Ammar Sohail
Moaz Ahmed
Shaheer Ahmar

DVSA attack surface is mostly service boundaries

Most findings came from how identity, events, permissions, and backend workflows crossed AWS services.

Privilege boundary

Lambda roles and admin routes need least privilege

Availability boundary

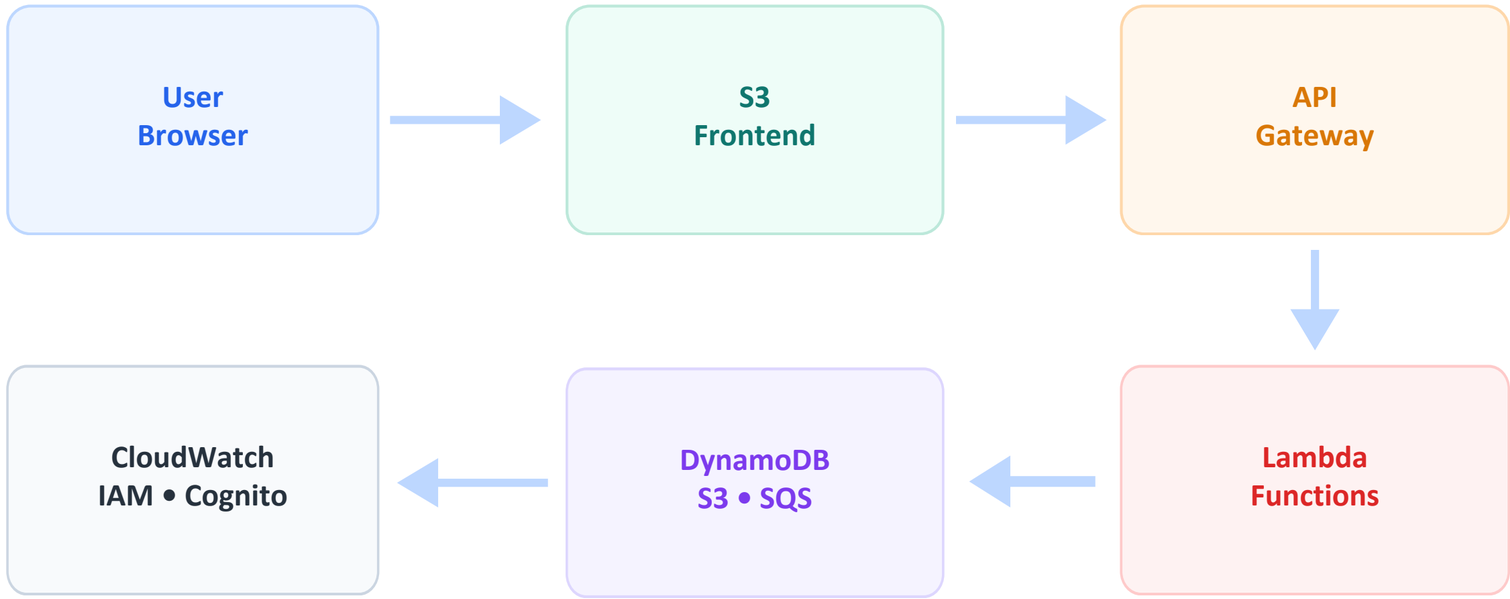
Expensive workflows need rate and concurrency controls

Input boundary

Unsafe payload parsing and malformed events

Identity boundary

JWT claims must be verified before use



Findings map: each lesson moved from exploit to fix

The common grading structure was: expected behavior, exploit proof, exact change, and post-fix verification.

Lesson	Weakness	Impact	Fix principle
1	**Event injection**	Lambda RCE	safe parsing
2	**Broken authentication**	user impersonation	JWT verification
3	**Sensitive data exposure**	receipt leak	authorized receipt access
4	**Insecure cloud config**	untrusted S3 trigger	bucket + key validation
5	**Broken access control**	mark paid as user	admin check
6	**Denial of service**	billing overload	throttling controls
7	**Over-privileged function**	large blast radius	least privilege IAM
8	**Logic vulnerability**	workflow abuse	state sequencing
9	**Vulnerable dependency**	unsafe library path	remove/replace package
10	**Unhandled exceptions**	internal detail leak	safe errors

RCE started with event injection and an unsafe dependency

A crafted order request reached unsafe deserialization in Lambda and executed attacker-controlled JavaScript.

Crafted /order event → node-serialize unserialize() → Lambda runtime executes code → CloudWatch proof

```
6a3ee1f3-5942-4a40-8af4-71be7c111c63 Version: $LATEST
28.641Z 6a3ee1f3-5942-4a40-8af4-71be7c111c63 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
4-71be7c111c63    ERROR    FILE READ SUCCESS: You are reading the contents of my hacked
```

```
// lesson 1 and 9
// var req = serialize.unserialize(event.body);
// var headers = serialize.unserialize(event.headers);
var req = typeof event.body === "string" ? JSON.parse(event.body) : (event.body || {});
var headers = event.headers || {};
```

Fix

Replace unsafe deserialization with safe JSON parsing and keep request fields as data.

Broken authentication came from trusting JWT payloads

The backend accepted identity claims after decoding the token instead of verifying the signature and required claims.

Attacker has valid token → Modify username/sub → Backend trusts payload claims → Victim orders returned

Exploit proof

- Only token claims changed; signature was not properly trusted.
- Orders API returned another user's order list and details.

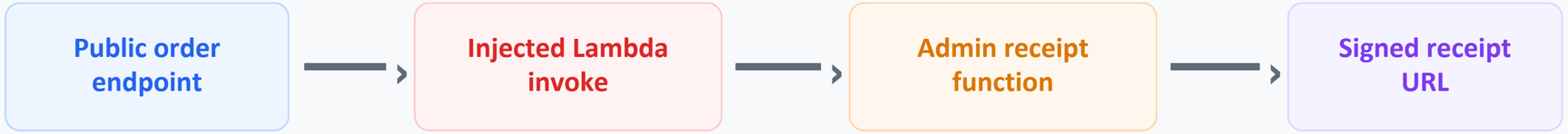
Remediation

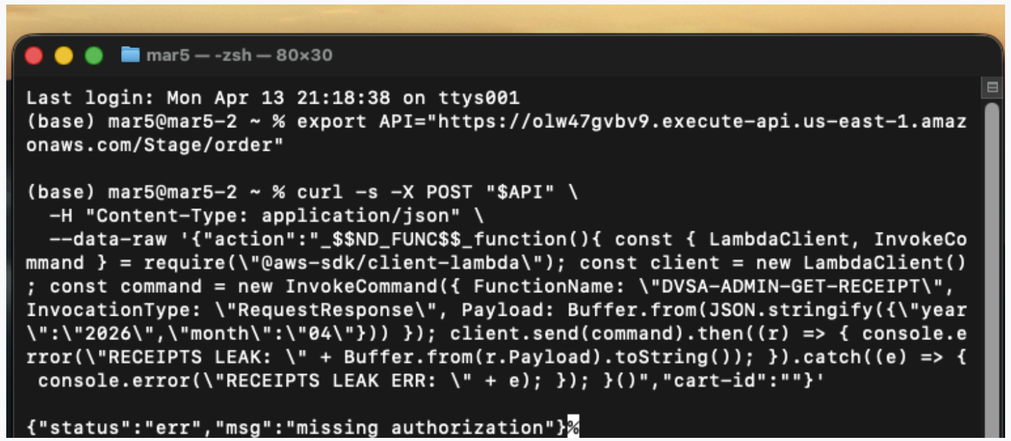
- Fetch Cognito JWKS and verify signature.
- Validate issuer, expiry, and token_use before reading identity.

```
(base) mar5@mar5-2 ~ % curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization:$FAKE_AS_C" \
  --data-raw '{"action":"orders"}' | jq

{
  "status": "err",
  "msg": "invalid token"
}
(base) mar5@mar5-2 ~ %
```

Receipt exposure showed why backend-only functions still need authorization





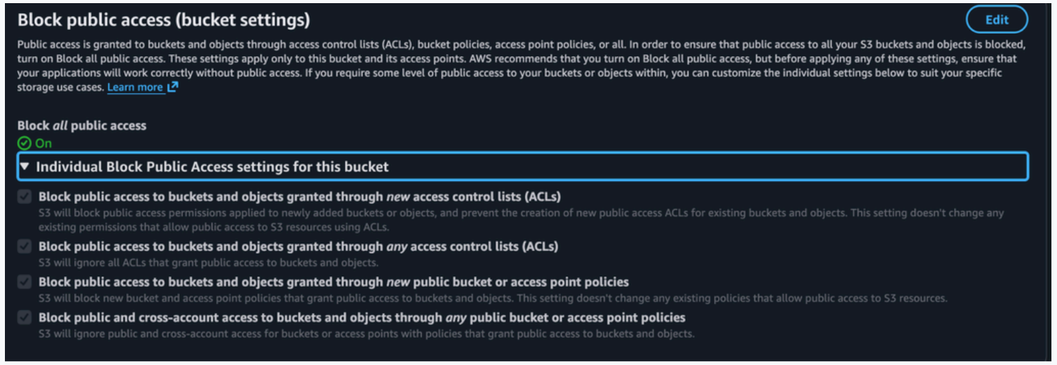
Abusing the injection path made privileged receipt-generation behavior reachable outside its intended boundary.

Security issue

Sensitive receipt-generation behavior was reachable through another vulnerable path.
Signed URLs must not be produced unless the caller is authorized for the receipt.
Blocking the injection path reduced exposure; the receipt function should also enforce its own authorization.

S3 configuration turned object storage into an event-driven attack surface

A manual .raw object upload triggered receipt-processing Lambda outside the normal application workflow.



send_receipt_email.py

```python
def lambda_handler(event, context):
    key = urllib.parse.unquote_plus(urllib.parse.unquote(key))
    #vulnerability
    # order = key.split("/")[3]
    # orderId = order.split("_")[0]
    # userId = order.split("_")[1].replace(".raw", "")

    #fix
    parts = key.split("/")
    if len(parts) != 4:
        print("Invalid receipt key format:", key)
        return {"status": "err", "msg": "invalid receipt key"}

    order = parts[3]

    if not order.endswith(".raw") or "_" not in order:
        print("Invalid receipt filename:", order)
        return {"status": "err", "msg": "invalid receipt filename"}

    orderId, userPart = order.split("_", 1)
    userId = userPart.replace(".raw", "")

    if not orderId or not userId:
        print("Missing orderId or userId:", order)
        return {"status": "err", "msg": "invalid receipt identifiers"}
```

Fix pattern: restrict who can write receipt objects, then validate object-key format before Lambda processes the event.

Broken access control bypassed the payment workflow

A regular user could reach the admin update path and mark an order paid without completing billing.


```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
  --function-name DVSA-ORDER-GET \
  --cli-binary-format raw-in-base64-out \
  --payload "{\"user\":\"${DVSA_USER}\",\"orderId\":\"${ORDER_ID}\",\"isAdmin\":\"false\"}" \
  --region "$AWS_REGION" \
  before.json

jq '.order.orderStatus, .order.orderId, .order.itemList, .order.address' before.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
100
"2361c30f-c2f4-4ae2-a972-986515693f7e"
{
  "1020": 1
}
"{}"
```

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
  --function-name DVSA-ADMIN-UPDATE-ORDERS \
  --cli-binary-format raw-in-base64-out \
  --payload file://admin-compact.json \
  --region us-east-1 \
  post-fix-exploit.json

jq . post-fix-exploit.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
{
  "status": "err",
  "msg": "invalid token"
}
```

Fix : enforce role authorization inside the privileged Lambda before any order update logic runs.

Availability was testable as a security property

The billing path was expensive enough that parallel requests could consume Lambda/API capacity and degrade checkout.

```
shaheer@DESKTOP-6I95C61:~$ aws lambda invoke \
  --function-name DVSA-ORDER-BILLING \
  --cli-binary-format raw-in-base64-out \
  --payload file://~/dvsa-billing-payload.json \
  ~/dvsa-billing-out-0.json

cat ~/dvsa-billing-out-0.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
{"status": "err", "msg": "order already made"}shaheer@DESKTOP-6I95C61:~$
shaheer@DESKTOP-6I95C61:~$ mkdir -p ~/dvsa-dos-out && rm -f ~/dvsa-dos-out/*
seq 1 40 | xargs -P 40 -I{} bash -c '
  aws lambda invoke \
    --function-name DVSA-ORDER-BILLING \
    --cli-binary-format raw-in-base64-out \
    --payload file://$HOME/dvsa-billing-payload.json \
    $HOME/dvsa-dos-out/out-{}.json >/dev/null
'
echo done

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): R
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries: 2): R
ate Exceeded.

Additional error details:
Type: User
```

```
Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.

Additional error details:
Type: User
Reason: ConcurrentInvocationLimitExceeded

aws: [ERROR]: An error occurred (TooManyRequestsException) when calling the Invoke operation (reached max retries
ate Exceeded.
```

Observed

TooManyRequestsException and ConcurrentInvocationLimitExceeded

HTTP effect

mixed 200/500/502 before tighter edge throttling

Mitigation

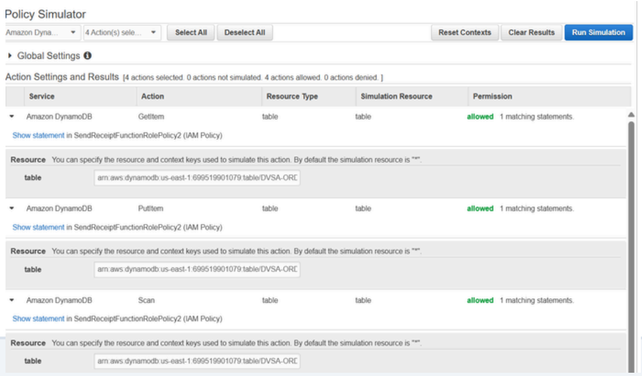
API Gateway throttling plus broader per-user and concurrency controls

Over-Privileged Functions

DVSA-SEND-RECEIPT-EMAIL role granted more AWS access than the receipt workflow required.

Evidence: broad policies attached to receipt function role

Evidence: Policy Simulator allowed S3 actions outside receipt workflow



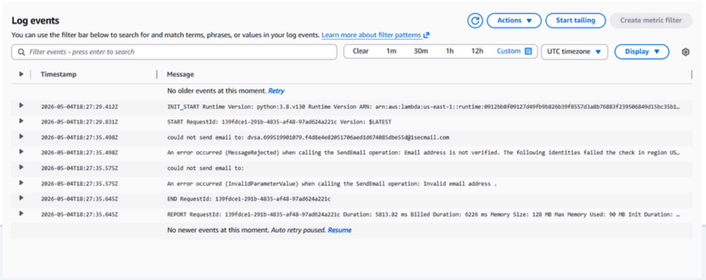
Finding

The Lambda role could access broad S3, DynamoDB, and SES resources.

Impact

If the function is abused, the attacker inherits those permissions and the blast radius expands beyond sending receipts.

Actual use: CloudTrail showed much narrower activity



Fix: remove wildcard policies and grant only required actions: s3:GetObject on the receipts bucket, dynamodb:GetItem on DVSA-ORDERS-DB, ses:SendEmail, and CloudWatch logging.

The remediation pattern was defense in depth

Each fix restored a broken security assumption at the layer where the trust decision actually happens.

1	**Validate input**	JSON/event schemas, object-key checks
2	**Verify identity**	JWT signature, issuer, expiry, token_use
3	**Authorize action**	admin checks before privileged updates
4	**Limit privilege**	narrow IAM roles and S3/DynamoDB scope
5	**Protect availability**	rate limits, quotas, idempotency, safe errors

Verification loop

Exploit → Fix → Retest → Normal path

Key takeaways for serverless security

The strongest controls were the ones enforced in backend and cloud layers, not only in the UI.

Frontend paths are not security boundaries

Users and attackers can call API Gateway directly.

Identity must be cryptographically verified

Decoded JWT payloads are data until signature and claims are validated.

Cloud configuration is application logic

S3 events, IAM roles, and Lambda triggers define real attack paths.

Fixes need proof

A remediation is complete only after the exploit fails and normal behavior still works.

Questions